

Звіт: DevOps-фреймворки (CALMS та інші) і SDLC

Навчальне завдання з дослідження. **Автор:** Андрій Пономарьов / Claude Code
Opus 4.8. **Дата:** 2026-06-25.

1. Assignment overview (опис завдання)

Завдання складалося з трьох частин:

1. **Ознайомитись** з матеріалом про фреймворк **CALMS** на сайті Atlassian: <https://www.atlassian.com/devops/frameworks/calms-framework>.
2. **Знайти інші фреймворки** (у сфері DevOps / організації розробки) і подати їх як список з коротким описом.
3. **Знайти, прочитати та резюмувати будь-який опис SDLC** (Software Development Life Cycle): навести посилання та основні тези.

Очікуваний результат — структурований звіт із посиланнями на джерела.

2. Initial analysis (початковий аналіз і припущення)

Це **дослідницьке (research) завдання**, а не написання коду. Тому «доказами виконання» тут є не скріншоти запуску програми, а **першоджерела (URL) та конспект прочитаного**. Усі ключові факти супроводжуються посиланнями.

Зроблені припущення:

- Під «іншими фреймворками» (п. 2) маються на увазі насамперед **DevOps-фреймворки та моделі організації розробки/експлуатації** (Three Ways, DORA, SRE тощо), а не випадкові програмні бібліотеки. Це логічно впливає з контексту п. 1 (CALMS — саме DevOps-фреймворк).
 - Для п. 3 дозволено взяти **будь-який** опис SDLC — обрано матеріал AWS («What is SDLC?») як стислий і авторитетний, плюс крос-перевірка з Atlassian.
 - Для повноти й перехресної перевірки фактів по кожній частині використано кілька авторитетних джерел (повний перелік — у розділі 8).
-

3. Частина 1 — Фреймворк CALMS

Джерело завдання: Atlassian — *CALMS Framework*

<https://www.atlassian.com/devops/frameworks/calms-framework> **Додаткові джерела:**
Sonatype <https://www.sonatype.com/blog/principle-based-devops-frameworks-calms>,
ManageEngine <https://www.manageengine.com/products/service-desk/itsm/devops-calms-framework.html>.

Що це таке

CALMS — це фреймворк (модель оцінювання), який допомагає організації зрозуміти, чи **готова вона до впровадження DevOps і наскільки просунулась у DevOps-трансформації**. Назва — акронім із п'яти стовпів. Авторство приписують **Jez Humble** (співавтор книг *The DevOps Handbook* і *Accelerate*).

П'ять стовпів CALMS

Літера	Стовп	Суть
C	Culture (Культура)	Руйнування «силосів» між Dev і Ops через культуру спільної відповідальності . Потрібен або готовий mindset команди, або «чемпіони» DevOps за підтримки менеджменту.
A	Automation (Автоматизація)	Автоматизація ручних задач, передусім CI/тестування . Основа — continuous integration / continuous delivery (CI/CD).
L	Lean (Ощадливість)	Lean-практики: усунення втрат і оптимізація потоку цінності — менше WIP (work-in-progress), більше прозорості, менше затримок і складних передач роботи.
M	Measurement (Вимірювання)	Систематичний збір даних про процеси й деплої, щоб розуміти поточні можливості та знаходити точки покращення. «Не можна покращити те, що не вимірюєш».
S	Sharing (Обмін)	Культура відкритості й обміну знаннями всередині та між командами; спільні цілі

Літера	Стовп	Суть
		та інструменти, що полегшують передачу роботи й розв’язання інцидентів.

Як використовується

CALMS працює як **чек-лист зрілості**: по кожному стовпу організація оцінює свій стан і визначає, де відстає. На практиці його часто поєднують із **DORA-метриками** (див. нижче): CALMS задає *напрямок* трансформації, DORA *вимірює результат*.

4. Частина 2 — Інші фреймворки (список з коротким описом)

Нижче — основні фреймворки та моделі, що використовуються в DevOps і організації розробки. Для кожного — стисла суть і де застосовується.

1. **The Three Ways (Три шляхи)** — філософська основа DevOps з книг *The Phoenix Project* / *The DevOps Handbook* (Gene Kim та ін.):
 - *Flow (Потік)* — прискорення руху роботи Dev → Ops → користувач (обмеження WIP, ефективні CI/CD-пайплайни);
 - *Feedback (Зворотний зв’язок)* — швидкі петлі від Ops назад до Dev, раннє виявлення збоїв;
 - *Continuous Learning (Безперервне навчання)* — культура високої довіри, експерименти й навчання на помилках.
2. **DORA / Four Key Metrics** (*DevOps Research and Assessment*, Google) — стандартизовані **метрики продуктивності** доставки ПЗ: *Deployment Frequency*, *Lead Time for Changes*, *Change Failure Rate*, *Time to Restore Service (MTTR)*. Дають бенчмарк «де ми порівняно з лідерами».
3. **SPACE Framework** — модель **продуктивності розробників**, ширша за DORA: *Satisfaction, Performance, Activity, Communication & collaboration, Efficiency & flow*. Підкреслює, що продуктивність не зводиться до однієї метрики.
4. **SRE (Site Reliability Engineering)** — підхід Google: застосування інженерних/програмних методів до експлуатації. Ключові поняття: **SLI/SLO/SLA**, **error budgets** (бюджети помилок), *toil reduction*, *blameless postmortems*.
5. **ITIL (IT Infrastructure Library)** — фреймворк **управління IT-послугами (ITSM)**: керування інцидентами, проблемами, змінами, релізами. Часто співіснує з DevOps у великих організаціях.
6. **Lean Software Development** — перенесення принципів Lean (Toyota) у розробку: усунення втрат, посилення навчання, пізніє прийняття рішень, швидка доставка, повага до людей, оптимізація цілого.

7. **Agile / Scrum / Kanban** — гнучкі методології розробки:
 - *Scrum* — ітерації-спринти, ролі (PO, SM, команда), церемонії;
 - *Kanban* — візуалізація потоку, обмеження WIP, безперервна доставка. DevOps часто будується «поверх» Agile.
8. **SAFe (Scaled Agile Framework)** — масштабування Agile на рівень великих організацій (програми, портфелі, ART — Agile Release Trains).
9. **Spotify Model** — організаційна модель: *squads, tribes, chapters, guilds*. Радше натхнення/приклад, ніж жорсткий фреймворк.
10. **DevSecOps** — інтеграція **безпеки** у весь конвеєр («shift left security»): безпека вбудовується з самого початку, а не «прикручується» наприкінці.
11. **Value Stream Management (VSM)** — управління наскрізним потоком цінності від ідеї до користувача; пошук «вузьких місць» у всьому ланцюжку доставки.

Підсумок порівняння: CALMS і Three Ways задають культуру й принципи; DORA та SPACE вимірюють; SRE й ITIL стосуються експлуатації/послуг; Agile/Scrum/Kanban/SAFe — процес розробки; DevSecOps і VSM — наскрізні аспекти (безпека та потік цінності). Вони не конкурують, а доповнюють одне одного.

5. Частина 3 — Резюме опису SDLC

Обране джерело: AWS — *What is SDLC? Software Development Lifecycle Explained*
<https://aws.amazon.com/what-is/sdlc/> **Крос-перевірка:** Atlassian
<https://www.atlassian.com/agile/software-development/sdlc>, GeeksforGeeks
<https://www.geeksforgeeks.org/software-engineering/software-development-life-cycle-sdlc/>.

Визначення

SDLC (Software Development Life Cycle) — це «**економічно та часово ефективний процес**, який команди розробки використовують для проєктування й створення якісного ПЗ». Він зменшує ризики через структуроване планування, щоб результат відповідав потребам замовника.

Фази (за AWS — 6 фаз)

1. **Plan (Планування)** — збір вимог від стейкхолдерів, аналіз «витрати/вигоди», оцінка ресурсів, документ SRS (Software Requirements Specification).
2. **Design (Проектування)** — аналіз вимог і вибір оптимальних технічних рішень (технології, архітектура, інтеграції).
3. **Implement (Реалізація)** — написання коду; робота розбивається на керовані щоденні задачі.
4. **Test (Тестування)** — комбінація автоматичного та ручного тестування для пошуку дефектів і перевірки відповідності очікуванням.

5. **Deploy (Розгортання)** — фінальна збірка переноситься з тестового середовища у production (живу систему, доступну користувачам).
6. **Maintain (Супровід)** — виправлення багів, реагування на звернення, моніторинг продуктивності, пошук можливостей покращення.

Примітка: різні джерела ділять процес на 5–7 фаз (Atlassian/GeeksforGeeks часто виокремлюють ще «Requirements Analysis» окремо від Planning). Суть однакова.

Основні моделі SDLC

- **Waterfall (Каскадна)** — послідовні фази; дисциплінована, але негнучка; підходить для невеликих проєктів із чітко визначеними вимогами.
- **Iterative (Ітеративна)** — повторювані цикли з інкрементами; гнучка, але є ризик «розповзання» обсягу (scope creep).
- **Spiral (Спіральна)** — поєднує ітерації з **аналізом ризиків**; добра для складних проєктів, але дорожча.
- **Agile** — короткі цикли з постійним зворотним зв'язком; ефективна, але вразлива до надмірних змін вимог.
- **V-Model** — варіація Waterfall, де кожній фазі відповідає своя фаза тестування (verification & validation).
- **DevOps / DevSecOps** — перетворює SDLC на **безперервний цикл** із CI/CD; DevSecOps вбудовує безпеку на всіх етапах.

Основні тези (чому SDLC важливий)

- Надає «**систематичний управлінський каркас із конкретними deliverables на кожному етапі**», що дозволяє стейкхолдерам узгодити цілі **наперед**.
 - Ключові вигоди: **краще управління ризиками**, точніша оцінка вартості, більша прозорість, вища задоволеність замовника.
 - Головна теза: SDLC перетворює **хаотичну розробку на керований і вимірюваний процес**, який адаптується під потреби організації.
-

6. Problems encountered (проблеми та рішення)

Проблема	Рішення
Різні джерела дають різну кількість фаз SDLC (5/6/7).	У звіті явно зазначено цю розбіжність і обрано модель AWS (6 фаз) як базову з приміткою.
Багато матеріалів змішують CALMS із суміжними поняттями (Three Ways, DORA).	Розведено: CALMS подано окремо (розділ 3), решту — у списку фреймворків (розділ 4) з поясненням, як вони співвідносяться.

7. Final result (підсумок)

Виконано всі три частини завдання:

-  **CALMS** опрацьовано — наведено визначення, авторство (Jez Humble), розшифровку всіх 5 стовпів і спосіб використання (оцінка зрілості DevOps).
 -  **Інші фреймворки** — наведено список з 11 пунктів із коротким описом і підсумковим порівнянням ролей.
 -  **SDLC** — обрано джерело (AWS), резюмовано визначення, 6 фаз, 6 моделей і основні тези; зроблено крос-перевірку.
-

8. Sources (усі джерела)

CALMS: - Atlassian — CALMS Framework:

<https://www.atlassian.com/devops/frameworks/calms-framework> - Sonatype — Principle-based DevOps frameworks: CALMS: <https://www.sonatype.com/blog/principle-based-devops-frameworks-calms> - ManageEngine — DevOps CALMS Framework: <https://www.manageengine.com/products/service-desk/itsm/devops-calms-framework.html>

Інші фреймворки: - Atlassian — DevOps Frameworks (огляд):

<https://www.atlassian.com/devops/frameworks> - Hatica — DORA vs SPACE: <https://www.hatica.io/blog/comparing-dora-vs-space-frameworks-for-developer-productivity/> - Jellyfish — DevOps Frameworks: <https://jellyfish.co/library/devops/frameworks/>

SDLC: - AWS — What is SDLC?: <https://aws.amazon.com/what-is/sdlc/> - Atlassian — What is SDLC?: <https://www.atlassian.com/agile/software-development/sdlc> - GeeksforGeeks — SDLC: <https://www.geeksforgeeks.org/software-engineering/software-development-life-cycle-sdlc/> - Wikipedia — Systems development life cycle: https://en.wikipedia.org/wiki/Systems_development_life_cycle